In-memory account ledger core — Part 2

Architectural decisions, trade-offs, and production considerations arising from the implementation at `github.com/kumiDa/ledger-core`*. Figures below are measured against that code, not estimated; the harness is described where each number appears.*

1. Append-only at scale

What breaks first at 100× volume, where state accumulates without bound, and the cheapest deferral.

The append path is not what fails. `Journal.append` is O(1) and the entry record is immutable, so writes scale until memory does. What fails is the **read path**, and it fails because every balance is derived. `Journal.balance` is a full scan of the journal filtered by account and the two dates, and the day-close loop calls it roughly three times per account per day — once for the fee trigger, once for the post-fee close, once more underneath the availability check. Scan cost grows with total entries while the number of scans grows with accounts times days, so replay is quadratic in volume.

Measured on synthetic streams of funded accounts over the same six-day window, scaling account count only:

Accounts	Entries	Replay (s)	Growth exponent
128	`6,144`	`0.39`	—
256	`12,288`	`1.29`	`1.73`
512	`24,576`	`4.62`	`1.84`
1,024	`49,152`	`18.50`	`2.00`

The exponent is *d*(log time)/*d*(log entries): 1.0 is linear, 2.0 quadratic. It reaches **2.00 at 1,024 accounts** — not approaching quadratic but arriving there. Profiling confirms the cause rather than inferring it: `Journal.balance` is 57% of total replay time at 128 accounts and **82% at 512**, and that share rises with scale because the scan it performs grows while everything around it does not.

Extrapolating the measured exponent from 1,024 accounts to 50,000 — roughly the 100× the question asks about — a close that takes 18.5 seconds becomes **about 12 hours**. The same work at linear cost would take 15 minutes. That gap is the whole problem: the close must fit inside an overnight window that does not grow with the book, and at quadratic cost it stops fitting long before the data stops fitting in memory.

Where state accumulates without bound

Three places, and the useful distinction is between unbounded *by design* and unbounded *by accident*. Sizes are measured by pickling each structure and normalising per event:

Structure	Per 1M events	Bounded?	Why it grows
Journal entries	36 MiB	By design	Append-only is the requirement. Must be retained.
Event results	118 MiB	By accident	One record per event, holding its detail string and entry tuple. Retained for the whole replay though nothing reads it after its day closes.
Holds	32 MiB	By accident	Released holds are never evicted; the dict keys on authorisation id for the lifetime of the run.

The event-result store is the largest of the three and is the one with no justification for existing in memory at all. It costs more than the ledger it describes, because it records *requests* — including declines and rejections that moved no money — at a wider record width than the movements themselves. That is a defensible audit artefact and an indefensible retention policy.

The holds dict carries a second, compounding cost. `active_hold_total` iterates every hold ever placed and filters for live ones, so availability-check cost tracks lifetime authorisation count rather than active count. Measured on one account with 2,400 lifetime holds of which 400 were still active: 175 µs per call, scanning 2,400 records to sum 400. Across the synthetic runs 83% of retained holds were dead weight, and that fraction rises with account age — an account open for a year carries a year of settled authorisations into every card swipe.

The cheapest structural change

Index entries by account on append — one dictionary of account id to entry list, maintained in `Journal.append`, with `balance` walking only the relevant account's list. Prototyped and measured against the committed engine on identical streams:

Accounts	Baseline (s)	Indexed (s)	Speed-up	Indexed exponent
128	`0.39`	`0.13`	`3.1×`	—
256	`1.29`	`0.25`	`5.2×`	`0.96`
512	`4.62`	`0.47`	`9.8×`	`0.93`
1,024	`18.50`	`0.96`	`19.3×`	`1.02`

This restores linearity — exponent 1.02 against the baseline's 2.00 — and the advantage widens with scale, from 3.1× at 128 accounts to 19.3× at 1,024, so even the largest measured figure understates the benefit at real volume. It is the cheapest change available because **it is an index, not a cache**: it derives nothing, so it cannot go stale, and in particular it is immune to the backdated entries that make caching hazardous here. No invalidation logic, no correctness surface, roughly fifteen lines.

It defers rather than solves. Balance remains O(entries per account), which is fine for a retail current account and wrong for a high-throughput merchant account. The next step is per-(account, value date) closing-balance snapshots, and that is where the real design hazard lives: a snapshot keyed on account alone is *wrong the moment a backdated entry arrives*. Invalidation must be scoped by value date — an entry value-dated V invalidates V and every later day, and nothing before it. Getting that wrong produces balances that are fast and false, which is strictly worse than slow and true. Evicting released holds and streaming event results out of memory are independent and similarly cheap.

2. Value-dated entries in production

Operational and regulatory surface in a UAE-licensed bank, and one control to add before go-live.

Separating posting date from value date is what makes the ledger correct, and it is also what turns a closed day into a mutable figure. Restatement is a reporting operation in the implementation; in a licensed bank it reaches into tax, disclosure, and supervisory reporting.

Operational surface

Statement finality. A backdated entry changes a balance the customer has already been shown. Under the CBUAE Consumer Protection Regulation and its accompanying Standards, statement accuracy and a defined error-resolution path are obligations, not best practice, so every restatement is a customer-communication event with a clock attached rather than a silent correction.

Fee and interest restatement. The implementation assesses overdraft fees forward-only: a day's fee is decided at that day's close and a later backdated debit never makes a closed day chargeable. The repository prices that choice at AED 50.00 on a six-day, two-account stream and leaves a test failing to keep the figure visible. The same policy across a portfolio is a systematic under-recovery, and a systematic fee error discovered after the fact is a remediation programme with regulatory reporting, not a code fix.

Tax periods. Banking fees are taxable supplies under the UAE VAT regime at the standard 5% rate. A fee value-dated into a tax period whose return has been filed cannot simply appear; it requires an adjustment or voluntary disclosure. The ledger has no concept of a tax period, so nothing in it can refuse that entry.

Supervisory and accounting close. Periodic returns to the CBUAE, and the general-ledger close behind them, are struck on dates. An entry value-dated before a submitted return restates a reported period. Interest recognition under IFRS 9 has the same exposure: the accrual basis for a closed day changes after the fact.

Reproducibility and retention. AML/CFT obligations require transaction records to be kept for at least five years. The requirement is not the current balance but the *as-of* view: what the books said on a given day, reconstructible years later. The implementation can answer this — its balance query takes both dates and nothing is ever mutated — which is the one part of this surface the design already handles well.

Business-day calendar. Days in the implementation are ordinals with no calendar, timezone or cutoff. UAE value dating must respect a Monday–Friday working week, public holidays, and a daily cutoff after which items carry the next business day. An ordinal day cannot express any of that.

The control to add before go-live

A hard backdating horizon enforced at the ledger boundary. Reject any entry whose value date precedes the last closed accounting period — a guard in `Journal.append`, the single chokepoint every entry already passes through — with a maker-checker break-glass that books the item into the *current* period and emits an explicit restatement record naming the original value date.

This control is the one to pick first because every other exposure above inherits its bound. A statement cannot be restated after issue, a filed VAT return cannot acquire a new fee, a submitted supervisory return cannot move, and closed-day interest cannot change — not because each path is individually defended, but because the entry that would trigger them cannot be written unilaterally. One guard, one invariant, and the blast radius of a value date becomes a known quantity instead of an open one.

Its cost should be stated honestly: genuine late-arriving items, such as correspondent-bank advices, get booked to the current period with a memo rather than their true economic date. That is a real loss of fidelity, and it is the trade banks actually make, because an unbounded backdating window is the larger exposure.

3. Authorisation lifecycle

Every terminal state other than a matching settlement, with the behaviour to mandate for each.

Six terminal states exist in the implemented model. Two more are missing, and the omissions matter more than the implementations.

Terminal state	Real-world scenario	Mandated behaviour

Declined on availability	Insufficient funds at authorisation time, including funds already encumbered by other holds.	No hold, no entry, decision recorded. As implemented.
Rejected, duplicate id	Network retransmission of an authorisation request.	Idempotent rejection; the live hold is never replaced or doubled. As implemented.
Settled under the hold	Fuel or hotel pre-authorisation, or partial fulfilment, where the final amount is lower.	Post the actual, release the hold in full, book nothing for the difference — it is returned availability, not a movement. As implemented.
Closed early by a first partial capture	Split shipment or partial fulfilment: a merchant captures against one authorisation more than once as goods leave the warehouse.	**Diverges from the implementation.** The model treats the first capture as an under-settlement and releases the hold, so every later capture against that authorisation is rejected and the remaining goods are never charged — measured at AED 40.00 unbilled on a 100.00 authorisation captured 60 then 40. Mandate multiple captures against a live authorisation, each *reducing* the hold rather than releasing it, until it is exhausted, voided, or expires.
Settled over the hold	Tip added after authorisation; final pump total above the pre-auth.	Accept — the money moved — but enforce a tolerance band (scheme-defined, typically a percentage or fixed cap). Above it, route to a review queue. The implementation accepts and flags without a band; the band is the production requirement.
Duplicate settlement	Acquirer resend or double-send of a capture.	Idempotent rejection, no second posting. As implemented.
Settlement with no authorisation	Force-post, offline-authorised transaction, or an auth that expired and was purged. Occasionally misrouting or fraud.	**Diverges from the implementation.** The engine rejects and posts nothing, which is correct for the brief. In production a force-post is a scheme obligation the bank will be held to: accept it to a suspense account and investigate. Rejecting it means the ledger disagrees with the scheme.

Terminal states the model cannot express

Expiry. There is no hold time-to-live. An approved authorisation that is never settled stays active for the life of the run — which is exactly what the brief's unsettled authorisation would have done had it been approved rather than declined. At portfolio scale this is not untidiness, it is *permanently encumbered customer funds*: money the customer cannot spend and the bank never took. Mandate a scheme-specific validity window (broadly seven days card-present, longer for some merchant categories), with release booked as an explicit, auditable expiry event — never a silent sweep, because a hold that vanishes without a record cannot be explained to a customer or an auditor.

Restoration after a reversed settlement. Reversing a settlement writes the contra-entry but does not reinstate the hold it consumed, so the funds silently become available again with no encumbrance behind a transaction that may still complete. Mandate that a settlement reversal either restores the hold for the remainder of its validity window or is explicitly recorded as declining to — never left implicit.

Void before capture. The model's reversal operates on ledger entries, and an authorisation produces none, so an authorisation cannot be cancelled — only settled or left open forever. The real scenario is routine: a merchant cancels an order, a customer abandons a booking. Mandate a distinct void event that releases the hold and is recorded as its own terminal state, rather than overloading reversal, whose semantics are tied to contra-entries.

Two further states are out of scope but worth naming: an account blocked or closed while holds are active, and incremental authorisation, where a hotel extends a stay and raises the hold against the same identifier.

4. What was cut, and the risk each cut defers

Simplifications made to stay in scope.

Simplification	Production risk deferred
No double-entry. The journal is single-sided: customer-account movements only, with no contra leg to fee income or interest expense.	**The most significant cut.** The bank's own position is absent, so no trial balance can be struck and nothing proves the books balance globally. A fee debits the customer and credits nothing. Every control that depends on a balancing identity — the primary defence against a posting defect — is unavailable.
No concurrency control. Replay is single-threaded and strictly ordered.	Two authorisations evaluated concurrently can both pass the availability check against the same funds and both be approved, overdrawing an account that the rule says cannot be overdrawn. Needs per-account serialisation or optimistic locking on an account version.
No event idempotency. Authorisation ids are unique and an entry is reversible once, but general event ids are not de-duplicated.	A redelivered event posts twice. Any at-least-once transport in front of the ledger — which is every real one — double-counts.

No persistence. In-memory only, per the brief.	No durability, recovery, or restart. Production needs an event log as the system of record, with the journal rebuilt from it and periodic snapshots.
No period close. Nothing marks a day or month as shut.	The backdating window is unbounded, which is the exposure Section 2 exists to close.
No calendar, timezone, or cutoff. Days are ordinals.	Value dates cannot respect working weeks, UAE public holidays, or end-of-day cutoffs, so they will not agree with the dates any counterparty uses.
No day-count convention. Interest is a flat per-day rate.	Real products specify ACT/360, ACT/365 or 30/360, and the choice changes every accrual. An unstated basis is a figure that cannot be reconciled against a product sheet.
No tax treatment. Fees are booked gross.	VAT on fees is absent from the ledger entirely, so the ledger cannot be the source for a return.
Derived balances, O(n) reads. Correctness bought at the cost of scan-per-query.	Quantified in Section 1: quadratic replay, with the index as the cheap deferral.
Single global rounding mode (half-up) and a **same-numeral fee across currencies**.	A per-product rounding policy is normal in retail banking. The BHD fee carries the AED numeral with no FX basis and is likely an order of magnitude too large; it is latent only because that account never overdraws.

Section 1 harnesses are reproducible against the repository. The per-account index was prototyped as a subclass and deliberately not committed, so the published engine remains the one the test suite describes.